MLOps Pipeline — AWS Deployment Results

At a glance

On AWS (ap-south-1, Mumbai), the pipeline detected drift twice in 6,000 live requests and retrained itself both times. The API ended up serving model v3 with no redeploy, and CloudWatch emailed the team through SNS.

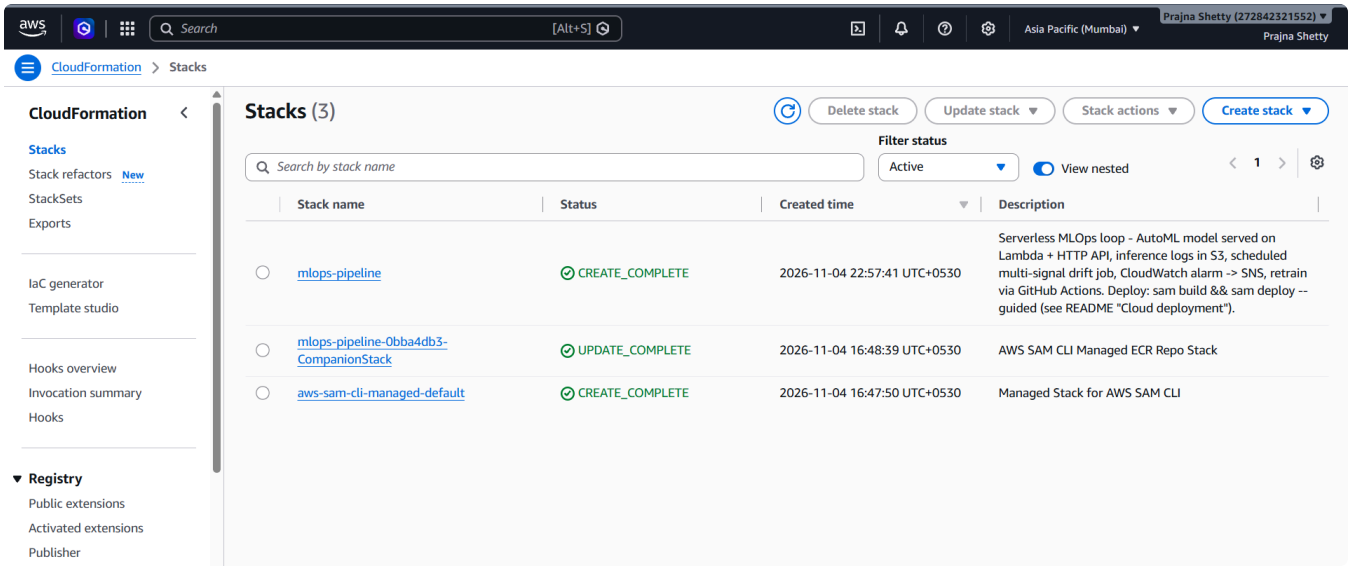
Version	Algorithm (picked by AutoML)	How it was created	Gate check
v1	Extra Trees	Seeded by invoking the drift Lambda with `{"action":"train"}`; 30 s AutoML search	93.2% validation accuracy
v2	XGBoost	Drift alarm after 3,000 requests (Elec2 rows 4,000–6,999); retrained in 13.0 s	83.0% vs 78.8% for v1, promoted
v3	XGBoost	Drift alarm after the next 3,000 requests (rows 7,000–9,999); retrained in 17.8 s	79.3% vs 73.0% for v2, promoted

The gate compares the new model with the serving model on the newest 20% of labelled data; a model is published only if it is at least as accurate.

Deployed infrastructure

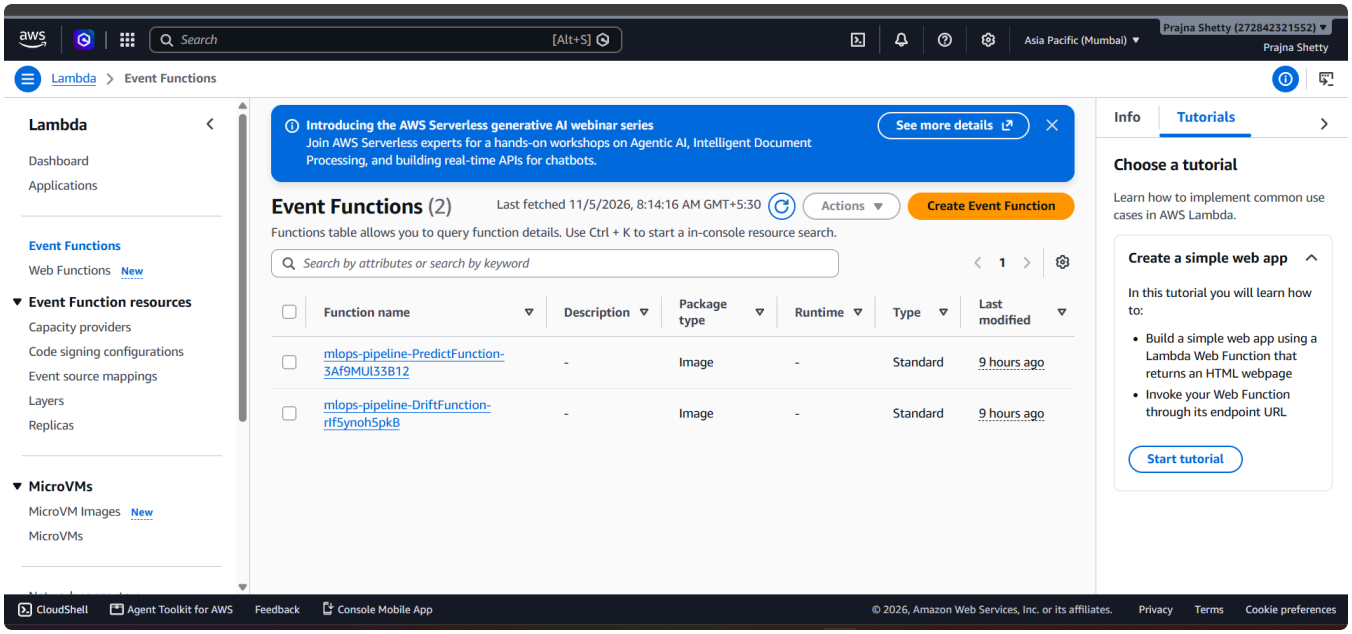
One `sam deploy` of the SAM template created the whole stack; nothing was set up by hand in the console.

Figure 1. CloudFormation stacks



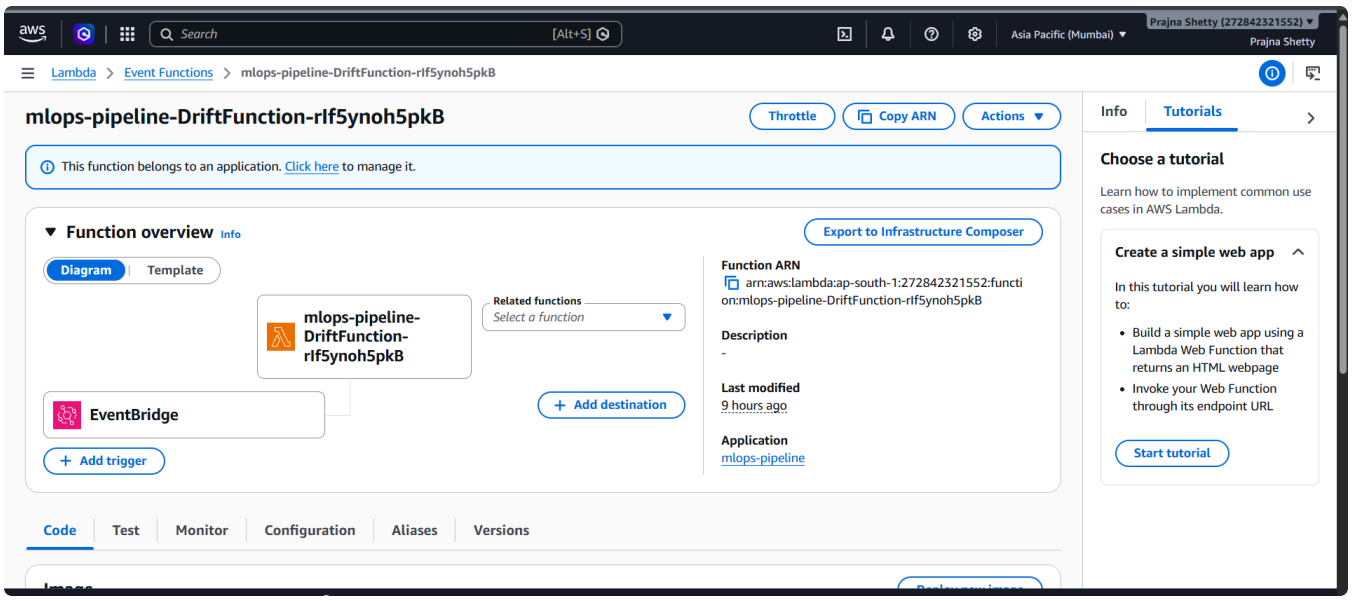
`mlops-pipeline` is the main stack (S3 bucket, both Lambdas, HTTP API, hourly schedule, CloudWatch alarm, SNS topic) and shows `CREATE_COMPLETE`. The companion stack holds the ECR repositories for the container images, and `aws-sam-cli-managed-default` is SAM's own upload bucket.

Figure 2. Lambda functions



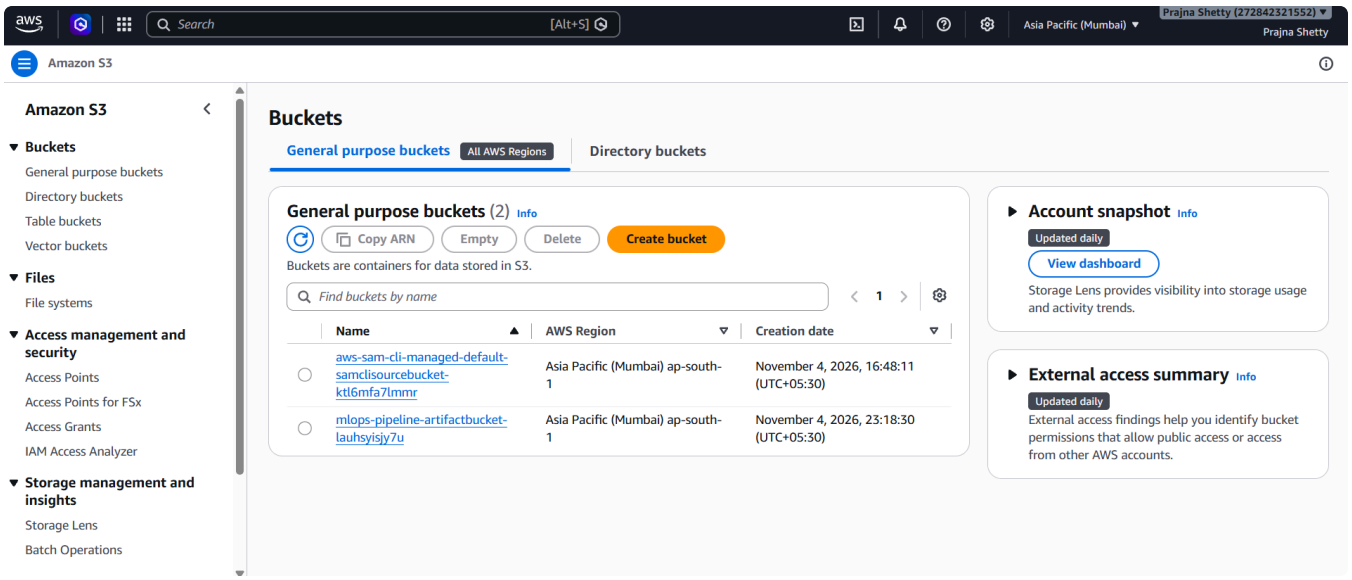
Two functions run the pipeline: `PredictFunction` serves the API and `DriftFunction` runs drift checks and retraining. Both are packaged as container images, because scikit-learn, LightGBM and XGBoost together exceed Lambda's 250 MB zip limit.

Figure 3. Drift function and its EventBridge trigger



The EventBridge trigger runs the drift function every hour automatically. During the demo it was invoked by hand only to avoid waiting for the next scheduled run.

Figure 4. S3 buckets



`mlops-pipeline-artifactbucket-…` is the pipeline's store: model versions, the `CURRENT` pointer, inference logs and labels. The other bucket belongs to SAM.

Closed-loop run

The loop ran end to end twice: live traffic in, drift detected, AutoML retrain, gate passed, new model served.

Figure 5. Seeding model v1

```
PS C:\mlops> $env:AWS_DEFAULT_REGION="ap-south-1"
PS C:\mlops> $fn = aws cloudformation describe-stack-resource --stack-name mlops-pipeline --logical-resource-id DriftFunction --query StackResourceDetail.Ph
ysicalResourceId --output text
PS C:\mlops> $fn
mlops-pipeline-DriftFunction-rIf5ynoh5pkB
PS C:\mlops> aws lambda invoke --function-name $fn --payload fileb://train.json --cli-read-timeout 900 out.json; Get-Content out.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}

{"version": 1, "estimator": "extra_tree", "val_acc": 0.9316666666666666, "seconds": 30.0807204246521}
PS C:\mlops>
```

Invoking the drift function with `{"action":"train"}` ran a 30 s AutoML search inside Lambda. It chose Extra Trees (`extra_tree`, 93.2% validation accuracy) and published it as version 1.

Figure 6. Two drift-and-retrain cycles

```
}

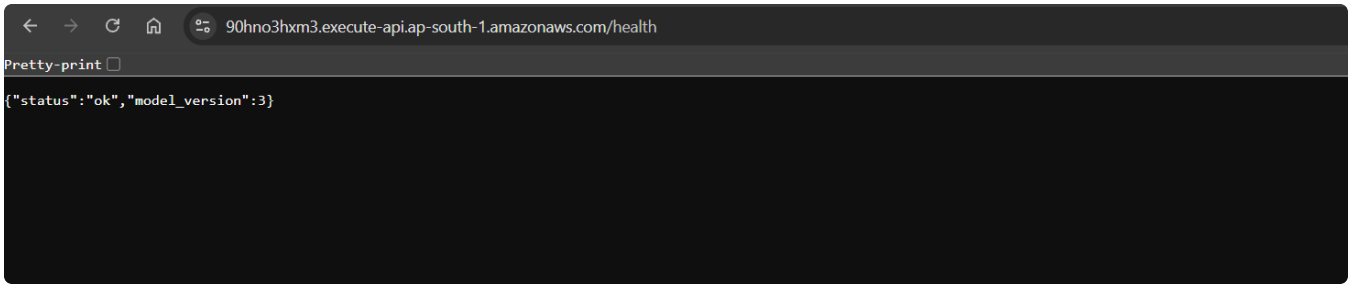
{"version": 1, "estimator": "extra_tree", "val_acc": 0.9316666666666666, "seconds": 30.0807204246521}
PS C:\mlops> $env:PYTHONWARNINGS="ignore"
PS C:\mlops> python experiments\replay_cloud.py https://90hno3hxm3.execute-api.ap-south-1.amazonaws.com 3000
health: {'status': 'ok', 'model_version': 1}
sent rows 4000..4050
sent rows 4000..4550
sent rows 4000..5050
sent rows 4000..5550
sent rows 4000..6050
sent rows 4000..6550
done: 3000 rows, model versions seen [1]
PS C:\mlops> aws lambda invoke --function-name $fn --payload fileb://check.json --cli-read-timeout 900 out.json; Get-Content out.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}

{"windows": 5, "alarms": [{"type": "retrain", "n_train": 3000, "acc_new": 0.83, "acc_old": 0.7883333333333333, "estimator": "xgboost", "promoted": true, "ve
rsion": 2, "seconds": 12.985665798187256}]}
PS C:\mlops> python experiments\replay_cloud.py https://90hno3hxm3.execute-api.ap-south-1.amazonaws.com 3000 7000
health: {'status': 'ok', 'model_version': 2}
sent rows 7000..7050
sent rows 7000..7550
sent rows 7000..8050
sent rows 7000..8550
sent rows 7000..9050
sent rows 7000..9550
done: 3000 rows, model versions seen [2]
PS C:\mlops> aws lambda invoke --function-name $fn --payload fileb://check.json --cli-read-timeout 900 out.json; Get-Content out.json
{
    "StatusCode": 200,
    "ExecutedVersion": "$LATEST"
}

{"windows": 4, "alarms": [{"type": "retrain", "n_train": 3000, "acc_new": 0.7933333333333333, "acc_old": 0.73, "estimator": "xgboost", "promoted": true, "ve
rsion": 3, "seconds": 17.769796133041382}]}
PS C:\mlops>
```

- `replay_cloud.py` sent 3,000 real Elec2 records to the live API, followed by their true labels; every request was answered by v1.
- The drift check found drift and retrained in 13.0 s. XGBoost scored 83.0% against v1's 78.8% on the newest data, so it was promoted to v2.
- The next 3,000 records (rows 7,000–9,999) were all answered by v2, showing the swap happened without a redeploy.
- A second drift check retrained again in 17.8 s: 79.3% against v2's 73.0%, promoted to v3.

Figure 7. Live API after both cycles

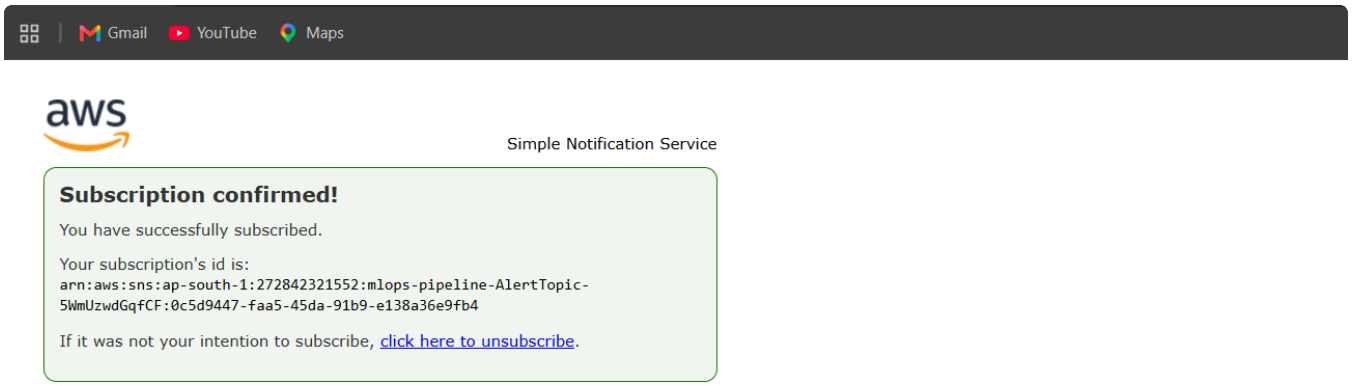


`/health` on the API Gateway URL reports `model_version: 3`. The API reread the `CURRENT` pointer in S3 and switched models by itself.

Monitoring and alerts

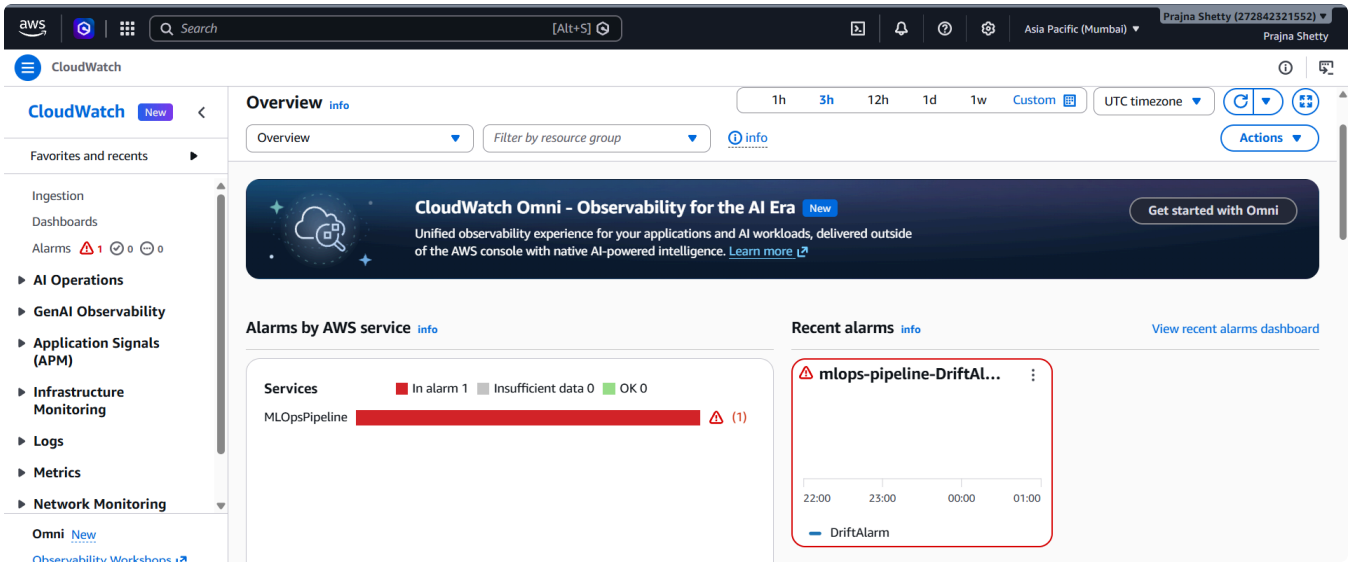
Each drift alarm also reached a human: the drift Lambda publishes a `DriftAlarm` metric, a CloudWatch alarm watches it, and SNS sends the email.

Figure 8. SNS subscription confirmed



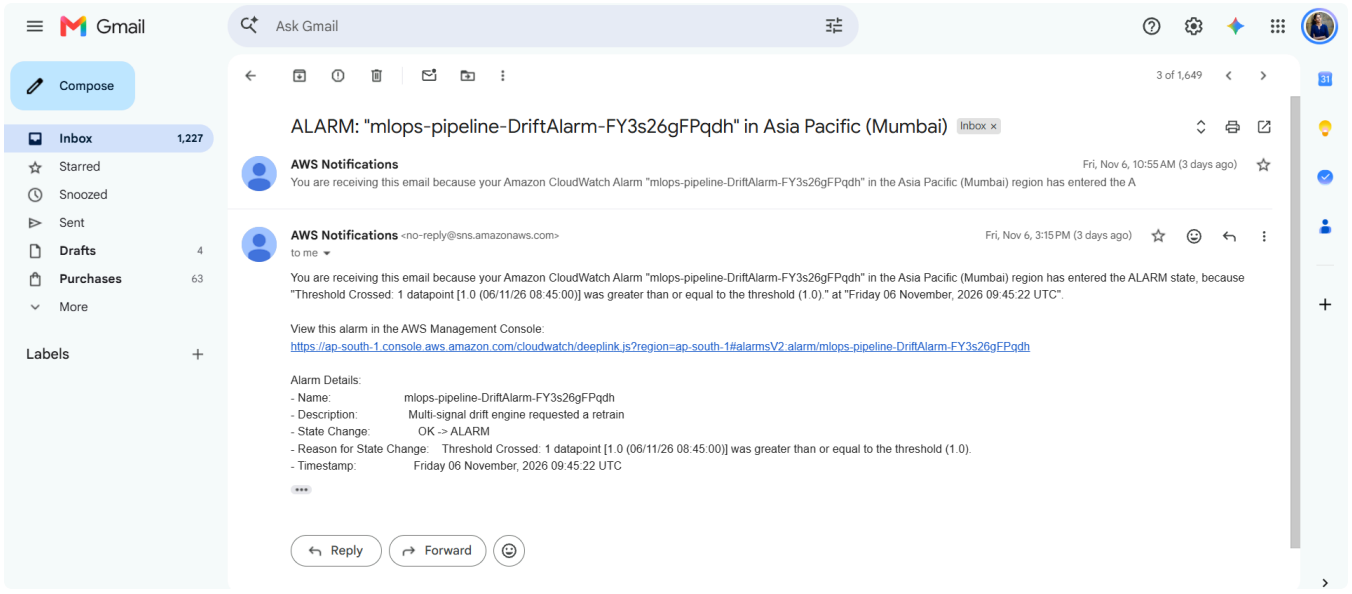
The team email was subscribed to the stack's `AlertTopic`. Without this confirmation SNS sends no alerts.

Figure 9. CloudWatch alarm in ALARM state



The `MLOpsPipeline` namespace shows one alarm in alarm: `mlops-pipeline-DriftAlarm`, which fires when the `DriftAlarm` metric reaches 1 within an hour.

Figure 10. Alert email from SNS



The email reports the state change from OK to ALARM: one datapoint of 1.0 crossed the threshold of 1.0 (06 Nov 2026, 09:45 UTC). It arrived while the retrain had already run automatically.